
Correcting Within-Group Self-Selection Bias in Prioritized Replay

Oscar Miró López-Feliu

University of Amsterdam

`oscar.miro.lopez.feliu@student.uva.nl`

Herke van Hoof

University of Amsterdam

`h.c.vanhoof@uva.nl`

Abstract

Prioritized experience replay (PER) improves sample efficiency by replaying high-priority transitions, usually according to absolute temporal-difference error. In stochastic environments, PER can distort the distribution of realized outcomes replayed from transitions with the same state-action pair. We call this within-group self-selection. We decompose PER into between-group allocation and conditional sibling selection, and derive corrections that preserve group-level prioritization at a fixed buffer and priorities: `SAMPLE` selects a group through PER and trains on a uniformly sampled sibling; `AVG` averages sibling Bellman targets; and `MODEL` samples from an empirical full-outcome model. In exact state-action environments with rare high-magnitude outcomes, sibling-aware replay improves learning efficiency over PER, with gains in return or success. We quantify how PER changes outcome frequencies and shifts mean Bellman targets within groups. In `MinAtar`, approximate VQ-VAE sibling groups combined with `SAMPLE` mitigate PER degradation under mean-preserving reward tails in four of five games, using learning rates selected on clean games. Matched PER-`SAMPLE` parameter sweeps show benefits under partial importance-sampling correction, while tuning can close the Bandit and TwoChains performance gaps; `FrozenLake` is more variable. Sibling-aware replay thus retains the focus on high-priority state-action regions while recovering their empirical outcome frequencies.

1 Introduction

Experience replay stores past transitions and reuses them for updates, reducing temporal correlation and improving data reuse in off-policy reinforcement learning [14]. In deep value learning, DQN combined replay with target networks to stabilize training from high-dimensional observations [15], while prioritized experience replay (PER) changed the replay distribution by sampling transitions according to priority, typically a power of absolute temporal-difference (TD) error [17]. Priority-driven replay has since become part of high-performing value-based agents such as `Rainbow`, `Ape-X`, and `R2D2` [5, 6, 8]. More recent analyses show that replay capacity, replay ratio, and buffer composition can substantially change learning outcomes [23, 3]. For PER, this makes the unit of prioritization important: transition-level priorities can change both which state-action regions are revisited and which stochastic outcomes dominate within those regions.

Non-uniform replay also changes the distribution of updates. PER samples individual transitions according to priority and applies importance-sampling (IS) weights to correct toward uniform replay [17]. In practice, this correction is often partial because the IS exponent is annealed toward one. Normalizing weights within the sampled minibatch can also change the expected update. Stale priorities and moving targets further affect learning dynamics [16]. Prior work studies the effective loss, coverage, and gradient-estimation consequences of non-uniform replay [4, 16, 11, 12]; other methods change priorities to reduce replay driven by irreducible or aleatoric noise [18, 2]. Together, these works show why transition priority can be useful yet imperfect as a proxy for learning value.

This paper isolates a conditional form of replay distortion. Consider a replay buffer containing several transitions with the same state-action pair (s, a) but different realized stochastic outcomes. We call these transitions *siblings*. PER performs two operations at once. First, it allocates more replay mass to some state-action groups than others; we call this *between-group allocation*. Second, within a selected group, it changes which realized outcomes are replayed more often; we call this *within-group self-selection*. Prior work mainly changes the transition priority score, and therefore changes the overall transition distribution or the amount of replay mass assigned to different regions. We instead keep the group-level priority mass and target the second effect: the conditional outcome frequencies used once a group has already been selected.

The first effect is often the desired behavior of PER, since some state-action regions may deserve more updates. The second effect can be harmful. If rare high-magnitude outcomes have persistently large TD errors for aleatoric reasons, then PER can replay the right region at the wrong conditional outcome frequencies. In that case, the problem is not only that some transitions receive too much replay probability globally, but that the empirical outcome distribution inside a fixed state-action group is tilted toward high-priority realizations.

We propose sibling-aware replay methods that separate PER’s group-level and within-group effects. At a fixed buffer and current priorities, the group-level priority mass is preserved, so high-priority state-action regions can still receive more replay effort. Within a selected group, the update is instead constructed from the empirical sibling distribution. SAMPLE trains on a uniformly sampled sibling transition; AVG averages sibling Bellman targets; and MODEL samples from the empirical full-outcome distribution for the exact state-action pair. Our contributions are: 1) We formalize a finite-buffer decomposition of PER into priority mass over state-action groups and priority-weighted sibling selection within each group. 2) We derive the induced sampling distribution and group-aware importance weights for SAMPLE, and we define AVG and MODEL as empirical estimators of the within-group Bellman target. 3) We evaluate the resulting methods in exact stochastic environments, where sibling groups are exact state-action groups, and in a function-approximation setting where frozen VQ-VAE codes provide approximate latent groups.

2 Related work

Experience replay and prioritized value learning. Replay reuses past experience [14] and became a core component of deep value learning through DQN [15]. PER replaces uniform transition sampling with priority-based sampling and importance weights [17]; it also appears in agents such as Rainbow, Ape-X, and R2D2 [5, 6, 8]. Replay capacity, replay ratio, and buffer composition can substantially affect learning [23, 3]. These findings motivate studying replay as an algorithmic sampling distribution; we examine its conditional structure within state-action groups.

Analyses of prioritized replay. Fujimoto et al. [4] relate non-uniform sampling to loss modification and analyze PER’s outlier bias in conditional value targets. Pan et al. [16] connect error-based prioritization to an effective loss and identify stale priorities and insufficient coverage as limitations. LaBER formulates replay sampling as a gradient-estimation problem [11], while Li et al. [12] analyze when TD error can proxy the value of replaying a transition. ROER derives replay reweighting from regularized occupancy optimization toward an optimal on-policy distribution [13]; DisCor reweights using estimated target accuracy to improve corrective feedback under bootstrapping [10]. These objectives differ from IS correction toward uniform replay. Our decomposition instead separates the probability of selecting a state-action group from the outcome distribution within that group.

Noisy, irreducible, and aleatoric transitions. High TD error can reflect noise rather than useful learning signal. ReLo uses online–target-network loss differences as a proxy for reducible loss [18]; UPER prioritizes information gain estimated from epistemic and aleatoric uncertainty [2]. These methods are close conceptual neighbors because they address when high-error transitions are useful replay targets. They change transition priority scores, which can affect both group allocation and conditional outcome selection. Our correction instead retains the current priority mass over groups and recovers empirical sibling frequencies within each selected group, without classifying error sources. The approaches are complementary: noise-aware priorities could improve group allocation, while sibling-aware sampling controls the outcome frequencies used within groups.

3 Finite-buffer decomposition of PER

Let a replay buffer contain N transitions

$$\mathcal{D} = \{z_i\}_{i=1}^N, \quad z_i = (s_i, a_i, r_i, s'_i, d_i), \quad (1)$$

where d_i denotes a terminal indicator. Let $p_i > 0$ be the raw priority, let $\alpha \geq 0$ be the priority exponent, and define the sampling mass $u_i = p_i^\alpha$. PER samples transition i with

$$P_{\text{PER}}(i) = \frac{u_i}{S}, \quad S = \sum_{k=1}^N u_k, \quad (2)$$

where S is the total priority mass in the buffer. For fixed priorities and a fixed per-transition loss $\ell_i(\theta)$, the idealized expected objective under unnormalized IS weights $w_i = (NP_{\text{PER}}(i))^{-\beta}$ is

$$\mathcal{J}_{\text{PER},\beta}(\theta) = \sum_{i=1}^N P_{\text{PER}}(i) w_i \ell_i(\theta) \propto \sum_{i=1}^N u_i^{1-\beta} \ell_i(\theta), \quad (3)$$

where $\beta \in [0, 1]$ controls the strength of correction and the omitted factor is independent of θ when priorities are held fixed [4, 16]. With unnormalized weights, $\beta = 1$ recovers the uniform gradient expected gradient, while $\beta < 1$ leaves residual priority weighting [17]. Our implementation divides weights by their maximum in the *sampled minibatch*. This random normalization generally changes the expected estimator, so the expected update can differ from uniform replay at $\beta = 1$.

Now define a fixed group key $\kappa(i)$. In the exact tabular setting, $\kappa(i) = (s_i, a_i)$. A group g contains the sibling indices

$$I(g) = \{i : \kappa(i) = g\}, \quad n_g = |I(g)|, \quad s_g = \sum_{j \in I(g)} u_j. \quad (4)$$

Here n_g is the number of observed siblings in group g , and s_g is their total priority mass. The PER sampling distribution decomposes as

$$P_{\text{PER}}(\kappa = g) = \frac{s_g}{S}, \quad P_{\text{PER}}(i \mid \kappa = g) = \frac{u_i}{s_g}, \quad i \in I(g). \quad (5)$$

Equation (5) separates two phenomena. The first term is PER's between-group allocation: some state-action groups receive more updates. The second is within-group sibling selection: once a group is revisited, high-priority realized outcomes are over-sampled relative to their empirical frequency. The empirical conditional sibling distribution is instead $P_{\mathcal{D}}(i \mid \kappa = g) = 1/n_g$.

Normalizing the expected weighted coefficients in Equation (3) within g gives

$$\pi_\beta(i \mid g) = \frac{p_i^{\alpha(1-\beta)}}{\sum_{j \in I(g)} p_j^{\alpha(1-\beta)}}. \quad (6)$$

This is an effective within-group weighting, not the actual conditional sampler, which remains p_i^α/s_g . Reducing α moves transition sampling toward uniform replay, affecting both components; increasing β changes update weights, leaving the sampler unchanged at fixed priorities. Both can indirectly change future priorities. At $\alpha = 0$, PER and SAMPLE both sample uniformly from the buffer. Appendix B details the assumptions and normalization distinction.

The same distinction appears in the update target. For any fixed scalar target T_i associated with transition i , the empirical sibling target for group g is the uniform average

$$\bar{T}_g = \frac{1}{n_g} \sum_{i \in I(g)} T_i, \quad (7)$$

whereas PER without IS correction ($\beta = 0$) uses the priority-weighted conditional mean

$$\tilde{T}_g^{\text{PER}} = \sum_{i \in I(g)} \frac{u_i}{s_g} T_i. \quad (8)$$

We call the gap between u_i/s_g and $1/n_g$ *within-group self-selection*. It is a finite-buffer sampling property, not by itself a claim about the true environment distribution. Uniform sibling sampling exactly recovers the empirical conditional distribution over observed siblings; it approximates the environment conditional distribution only insofar as the buffer is representative. The pathology appears when priority is driven by rare stochastic outcomes rather than reducible learning error: PER then biases the update toward high-priority realizations inside the group, rather than the empirical outcome frequencies.

Scope. All exact statements below assume a fixed buffer, fixed priorities, and fixed target parameters. They characterize the stochastic update distribution induced by replay. The decomposition applies to any fixed key map κ ; our exact claims use $\kappa(i) = (s_i, a_i)$, while the function-approximation experiments later use latent keys of the form $\kappa_\phi(i) = (\phi(s_i), a_i)$ for a frozen abstraction ϕ . Latent keys correct only within the chosen abstraction and can introduce abstraction error.

4 Sibling-aware replay

The methods below preserve PER’s group-selection role but change the outcome used inside the selected group. Algorithmically, PER samples an anchor transition A , the group $g = \kappa(A)$ is identified, and the within-group rule constructs the transition or target used in the update. Appendix B gives the corresponding finite-buffer derivations.

4.1 SAMPLE: uniform sibling replacement

SAMPLE treats the PER anchor only as a group selector. After drawing $A \sim P_{\text{PER}}$, it samples the trained index J uniformly from the anchor’s sibling group $I(\kappa(A))$ and trains on the full transition z_J . Thus, for any trained index j , writing $g = \kappa(j)$,

$$q_{\text{SAMPLE}}(j) = \Pr(J = j) = \sum_{i \in I(g)} \frac{u_i}{S} \frac{1}{n_g} = \frac{s_g}{S n_g}. \quad (9)$$

The finite-buffer IS ratio from this sampler back to uniform replay is

$$\rho_j = \frac{1/N}{q_{\text{SAMPLE}}(j)} = \frac{n_g S}{N s_g}, \quad w_j^{\text{grp}} = \rho_j^\beta. \quad (10)$$

At $\beta = 1$, fixed priorities, and fixed per-transition losses, SAMPLE with *unnormalized* group-aware IS matches the uniform-buffer expected estimator. For $\beta < 1$, residual weighting remains across groups, but siblings are uniform within each group for every β . Group weights, including their minibatch maximum, depend only on selected groups and therefore do not reintroduce conditional sibling weighting. These are fixed-buffer identities: different trained transitions and priority refreshes can make group masses diverge between methods during learning.

4.2 AVG and MODEL

In our value-based experiments, let

$$Y_j = r_j + \gamma(1 - d_j)Q_{\bar{\theta}}\left(s'_j, \arg \max_a Q_{\theta}(s'_j, a)\right) \quad (11)$$

be the Double-DQN target for sibling j [21]. AVG is a target-level correction: it replaces the sampled target by an average of sibling Bellman targets,

$$\hat{Y}_{g,K} = \frac{1}{K_g} \sum_{j \in U_K(g)} Y_j, \quad K_g = \min(K, n_g), \quad (12)$$

where $U_K(g)$ is a uniformly sampled subset of K_g siblings from $I(g)$. With the full group, AVG gives the empirical conditional Bellman target for g ; with a uniform subset, it is an unbiased finite-population estimator of that target.

MODEL instead samples from an empirical full-outcome distribution for each exact state-action group. It stores counts over tuples

$$Z = (s', r, d_{\text{term}}, d_{\text{trunc}}) \quad (13)$$

and samples a full outcome tuple when the corresponding exact group is selected. Sampling the joint tuple preserves dependencies between reward, next state, termination, and truncation. MODEL is directly applicable only in settings where exact state-action groups and full outcome tables are meaningful. For exact groups in the same buffer, MODEL and SAMPLE have the same conditional outcome distribution; their different priority-refresh rules can lead to different learning trajectories. Both AVG and MODEL use the group-aware IS weights in Equation (10), normalized by their minibatch maximum.

Priority refresh. Each method refreshes priorities after the optimizer step, using $|\delta| + \varepsilon$ from the TD errors computed before that step. PER refreshes its anchor; SAMPLE refreshes the trained sibling; MODEL refreshes the anchor using its resampled outcome. AVG refreshes the anchor or the siblings used in its target average. Appendix C.1 specifies the complete refresh rules.

4.3 Approximate latent siblings

The exact methods use $\kappa(i) = (s_i, a_i)$. For continuous, high-dimensional discrete, or image-based domains, exact state equality is usually too sparse or semantically brittle to define useful sibling groups. We therefore also consider a fixed abstraction ϕ and define latent keys

$$\kappa_\phi(i) = (\phi(s_i), a_i). \quad (14)$$

In the function-approximation setting considered here, ϕ is a frozen VQ-VAE encoder trained on random-policy observations [20]. It maps each observation to a packed discrete code grid. The same finite-buffer identities apply relative to these fixed abstract groups, but the correction no longer corresponds to the exact conditional outcome distribution for a true state-action pair. We select the abstraction using support diagnostics: code usage, observed $(\phi(s), a)$ buckets, bucket-size quantiles, fraction of transitions in sufficiently supported buckets. Appendix D and Table 9 report the support diagnostics and selected abstractions.

5 Experiments

We first evaluate sibling-aware replay in exact stochastic environments, then test approximate sibling groups in MinAtar. We report evaluation return and success, normalized area under the evaluation curve (AUC), and late-window performance. AUC summarizes learning efficiency over training; late-window metrics summarize performance near the end of training. We also measure conditional outcome distortion and descriptive replay statistics, defined in Appendix C.3. Exact-environment experiments use ten seeds; MinAtar uses five. All methods use mean-squared TD error; Appendix B.6 explains the distinction from Huber loss.

Exact environments. We use two custom diagnostic environments and one gridworld family. OutlierBandit has one state and two actions: a safe action gives 2, while a risky action gives 100 with probability 0.01 and 0 otherwise. The risky action has lower expected value (1 versus 2), but its rare reward can produce large TD errors. TwoChains has a safe branch returning 1 after eleven steps and a risky branch returning 80 with probability 0.01 after two steps, otherwise 0. FrozenLake uses the Gym/Gymnasium gridworld interface [1, 19], an 8×8 slippery map, intended-action probability 0.99, step reward -0.01 , and a 200-step limit. FrozenLake-H100/H300 uses goal reward $+100$ and hole reward -300 ; FrozenLake-H50 uses $+50/-100$ as a milder setting.

MinAtar. MinAtar provides compact Atari-like visual environments with lower computational cost than full Atari [22]. We use Asterix, Breakout, Freeway, Seaquest, and SpaceInvaders. Clean MinAtar is a control condition. The tail condition is MP-TAIL: for every positive training reward r , we replace it by $r - 30$ with probability 0.02 and by $r + 0.02 \cdot 30/0.98$ otherwise. This preserves the expected reward while introducing rare high-magnitude negative outcomes. Evaluation is always performed on the unperturbed environment. Latent SAMPLE uses frozen VQ-VAE state codes paired with actions as approximate sibling keys.

5.1 Exact state-action groups

Figure 1 and Table 1 compare the methods with $\alpha = 0.6$ and β annealed from 0.4 toward 1. In OutlierBandit, all methods eventually learn the safe action, but PER has substantially lower AUC.

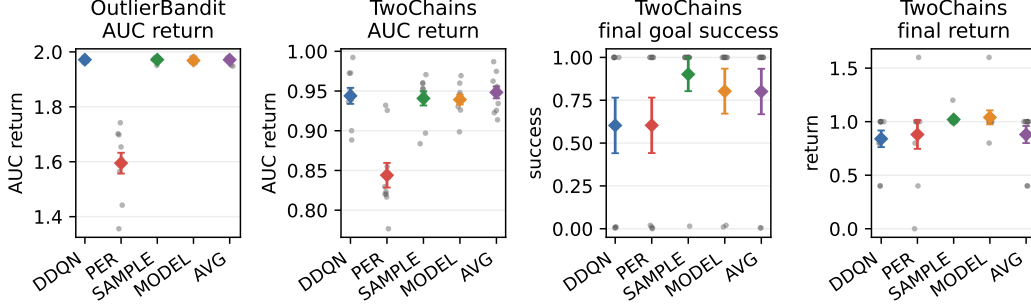


Figure 1: Exact-group results ($\alpha = 0.6$, annealed β). Grey points show seeds; colored points show mean $\pm$ SE. Each method uses ten seeds. TwoChains success counts goals from either branch.

Table 1: Exact-environment metrics, mean $\pm$ standard error over ten seeds. For OutlierBandit and TwoChains, the final columns are final evaluation metrics. For FrozenLake-H100/H300, the final columns are late-window metrics over the final 25k environment steps.

Env.	Method	AUC ret.	AUC succ.	Late/final ret.	Late/final succ.
Bandit	DDQN	1.97 ± 0.00	0.98 ± 0.00	2.00 ± 0.00	1.00 ± 0.00
	PER	1.59 ± 0.04	0.61 ± 0.03	2.00 ± 0.00	1.00 ± 0.00
	SAMPLE	1.97 ± 0.00	0.98 ± 0.00	2.00 ± 0.00	1.00 ± 0.00
	MODEL	1.97 ± 0.00	0.98 ± 0.00	2.00 ± 0.00	1.00 ± 0.00
	AVG	1.97 ± 0.00	0.98 ± 0.00	2.00 ± 0.00	1.00 ± 0.00
TwoChains	DDQN	0.94 ± 0.01	0.66 ± 0.05	0.84 ± 0.08	0.60 ± 0.16
	PER	0.84 ± 0.02	0.19 ± 0.06	0.88 ± 0.13	0.60 ± 0.16
	SAMPLE	0.94 ± 0.01	0.68 ± 0.03	1.02 ± 0.02	0.90 ± 0.10
	MODEL	0.94 ± 0.01	0.66 ± 0.03	1.04 ± 0.07	0.80 ± 0.13
	AVG	0.95 ± 0.01	0.69 ± 0.03	0.88 ± 0.08	0.80 ± 0.13
FrozenLake	DDQN	8.81 ± 2.99	0.44 ± 0.02	67.71 ± 3.83	0.863 ± 0.015
	PER	-8.81 ± 4.69	0.15 ± 0.03	55.72 ± 7.58	0.617 ± 0.075
	SAMPLE	4.69 ± 3.86	0.40 ± 0.02	72.10 ± 3.71	0.871 ± 0.016
	MODEL	12.89 ± 3.54	0.43 ± 0.02	72.08 ± 5.17	0.883 ± 0.018
	AVG-g2	2.64 ± 4.13	0.36 ± 0.03	69.22 ± 4.10	0.832 ± 0.021

In TwoChains, SAMPLE, AVG, and MODEL achieve higher mean final goal success than PER. Success includes goals reached through either branch. Section 5.3 examines sensitivity to the replay parameters.

In FrozenLake-H100/H300, PER learns more slowly under this schedule (Figure 2). SAMPLE and MODEL recover much of its lost late-window return and success (Table 1). Compared with DDQN, SAMPLE has higher mean late return and similar late success, while DDQN has higher mean AUC. Thus, the relative performance depends on whether we measure learning efficiency or the final training window.

To measure conditional outcome selection directly, we pool sibling entries that realize the same outcome o . Let $q_g(o) = \sum_{i \in I(g): Z_i=o} u_i / s_g$ be PER’s conditional outcome probability and $f_g(o) = n_{g,o} / n_g$ its empirical buffer frequency. Their total variation (TV) distance is

$$D_g^{\text{out}} = \frac{1}{2} \sum_o |q_g(o) - f_g(o)|. \quad (15)$$

SAMPLE has $D_g^{\text{out}} = 0$ on any fixed buffer. Appendix C.3 distinguishes outcome distortion, entry concentration, and IS-weight dispersion.

5.2 Approximate siblings in MinAtar

Table 2 reports the MP-TAIL AUC results using each method’s learning rate selected on clean games. Figure 3 compares clean and MP-TAIL learning curves. Under MP-TAIL, latent SAMPLE improves

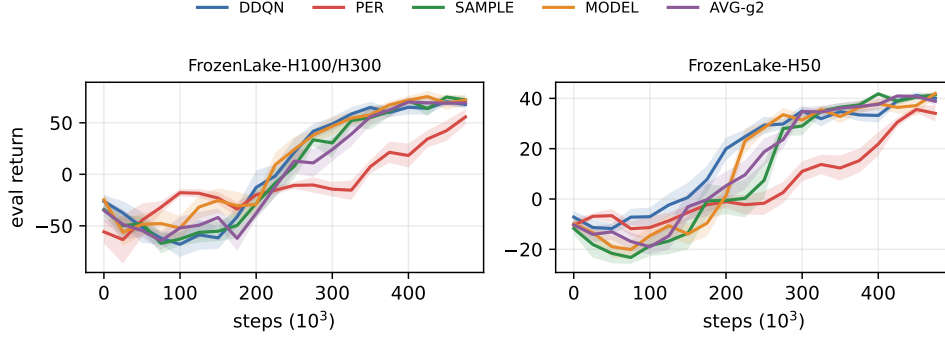


Figure 2: FrozenLake evaluation return curves. H100/H300 is the main exact gridworld test; H50 is a milder robustness setting. Shaded regions show standard error over ten seeds.

Table 2: MinAtar MP-TAIL AUC return, mean $\pm$ standard error over five seeds. Last columns report paired differences relative to SAMPLE, so positive values indicate that SAMPLE has higher AUC.

Game	DDQN	PER	SAMPLE	S-PER	S-DDQN
Asterix	22.07 ± 0.75	6.69 ± 0.39	22.80 ± 0.64	$+16.11 \pm 0.78$	$+0.73 \pm 0.50$
Breakout	13.72 ± 0.14	14.69 ± 0.23	14.56 ± 0.21	-0.12 ± 0.14	$+0.85 \pm 0.31$
Freeway	49.14 ± 0.49	1.80 ± 0.10	38.57 ± 0.90	$+36.77 \pm 0.89$	-10.57 ± 0.94
Seaquest	26.74 ± 1.55	2.12 ± 0.12	25.10 ± 2.39	$+22.98 \pm 2.37$	-1.64 ± 2.71
SpaceInv.	73.87 ± 1.72	19.62 ± 0.63	75.64 ± 0.67	$+56.02 \pm 1.04$	$+1.77 \pm 1.51$

over PER in Asterix, Freeway, Seaquest, and SpaceInvaders. The effect is largest in Freeway and SpaceInvaders, where PER largely fails under the tail perturbation while SAMPLE remains close to the stronger baselines. In Breakout, PER and SAMPLE are close on AUC and PER has the higher final return. The benefits therefore vary across games when transferring clean-selected learning rates to reward tails.

Clean MinAtar provides a control condition (Figure 3, top; Table 10). PER remains competitive with DDQN in Breakout and Seaquest, while DDQN is stronger in Freeway and SpaceInvaders. SAMPLE’s relative performance also varies across clean games. The MP-TAIL condition adds high-magnitude stochastic rewards while preserving their mean, allowing us to examine how the methods respond to reward tails.

DDQN is a strong baseline: under MP-TAIL, the smaller positive mean SAMPLE-DDQN AUC differences in Asterix, Breakout, and SpaceInvaders are uncertain, as is the difference in Seaquest, while Freeway favors DDQN (paired 95% t intervals; Appendix C.1). Together with the exact-group results, these comparisons support SAMPLE as a way to retain prioritized group selection while correcting conditional outcome frequencies. Appendix C.4 reports the descriptive replay diagnostics.

5.3 Sensitivity to replay parameters

We test whether ordinary PER tuning mitigates the performance gap by comparing PER and SAMPLE at every combination of $\alpha \in \{0.1, 0.3, 0.6, 1.0\}$ and fixed $\beta \in \{0, 0.4, 0.7, 1.0\}$ in all three exact environments. Each pair uses the same seed and training settings. Figure 4 summarizes ten paired seeds per setting; Appendix C.2 gives the full settings and all 48 comparisons.

SAMPLE shows clear gains under partial correction in Bandit and TwoChains. At $(\alpha, \beta) = (0.6, 0.4)$, the paired SAMPLE-PER AUC differences are 0.954 ± 0.017 in Bandit and 0.128 ± 0.008 in TwoChains. At $(0.6, 1)$ they shrink to -0.007 ± 0.004 and 0.001 ± 0.005 . Across both environments, $\beta = 1$ largely closes the gap; $\alpha = 0.1$ also does so in Bandit. Tuning can therefore recover similar mean performance while SAMPLE directly controls conditional selection.

FrozenLake is more variable: the paired AUC difference is 21.36 ± 5.71 at $(0.6, 0.4)$ and 1.90 ± 3.99 at $(0.6, 1)$, with a reversal of -9.57 ± 5.07 at $(1, 1)$. SAMPLE has negative mean AUC throughout the $\alpha = 1$ row. The simultaneous 95% interval at $(0.6, 0.4)$ is $[-5.75, 48.48]$, so this positive mean

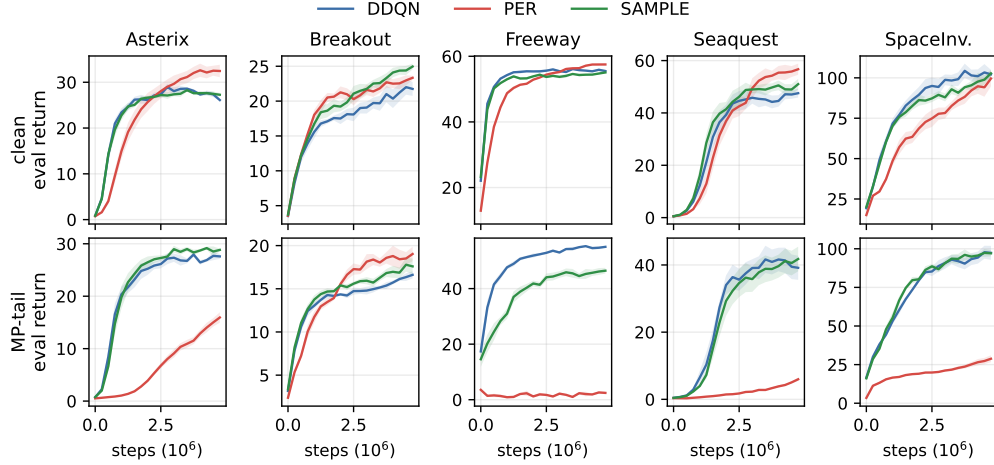


Figure 3: MinAtar evaluation return curves for clean training (top) and MP-TAIL training (bottom). Evaluation always uses the unperturbed environment; the mean-preserving tail is applied only during training. Shaded regions show standard error over five seeds.

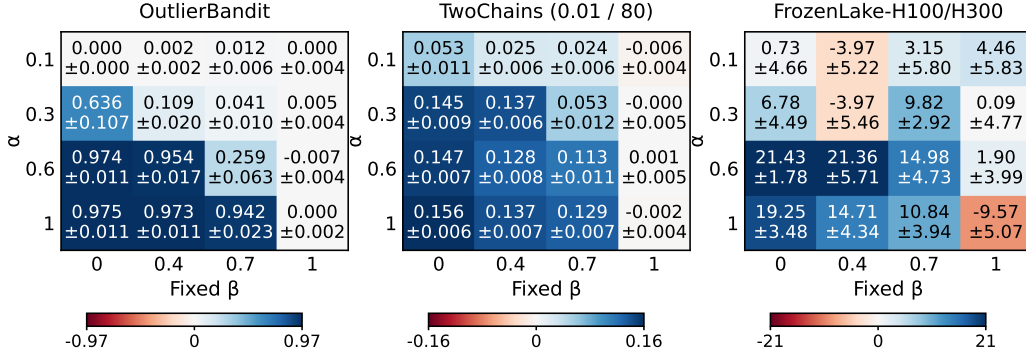


Figure 4: Matched SAMPLE-PER AUC differences, mean $\pm$ SE of ten paired seeds at all 48 fixed (α, β) cells across three environments. Positive values favor SAMPLE. Colors use separate reward scales per environment; SE summarizes variation across paired seeds.

remains uncertain. Conditional correction does not remove sensitivity to group allocation and update weighting.

The outcome measurements in Figure 5 clarify how tuning affects the mechanism. Larger α produces greater mean outcome distortion in these runs. Increasing β reduces the target shift under unnormalized IS, reaching zero at $\beta = 1$, while sampling distortion persists. This agrees with the fixed-buffer distinction: β changes update weights, whereas SAMPLE directly controls conditional selection. Appendix C.3 gives the aggregation details and measurements under the implemented weight normalization.

6 Discussion and conclusion

We studied a conditional failure mode of PER: transition-level priorities can over-select stochastic outcomes inside a state-action group. Our finite-buffer decomposition separates group allocation from sibling selection, allowing sibling-aware replay to retain prioritized group selection while recovering empirical outcome frequencies. Exact-group experiments and direct outcome measurements support this mechanism, and approximate MinAtar groups show its practical potential. The parameter sweep qualifies the performance gains: ordinary tuning can close the Bandit and TwoChains gaps, while FrozenLake remains more variable. SAMPLE controls conditional selection directly, independently of the chosen α/β .

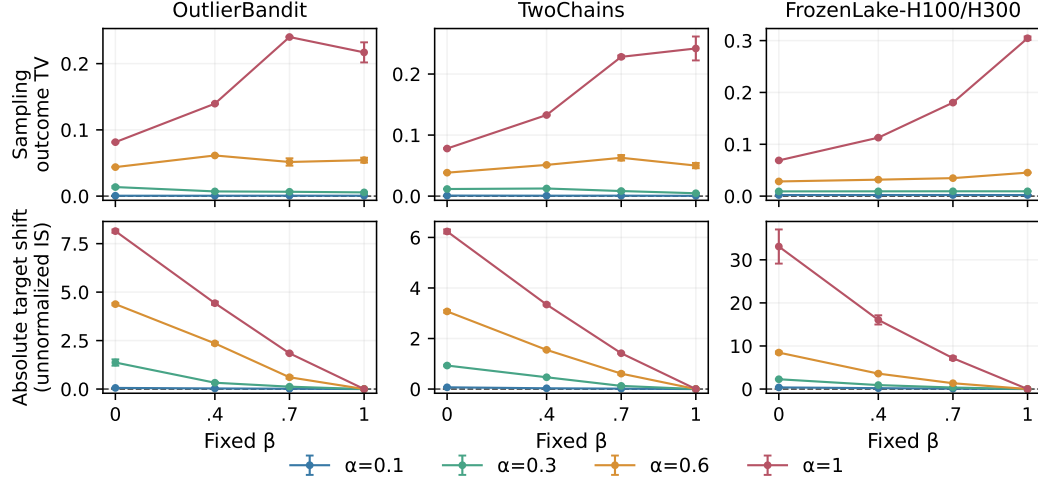


Figure 5: Full-buffer grid diagnostics, mean $\pm$ SE over ten seeds. Curves: PER; dashed zero: SAMPLE’s conditional identity. Top: outcome TV. Bottom: absolute conditional target shift under unnormalized IS, distinct from minibatch-max normalization. Both average groups by sampling mass s_g/S , then checkpoints within seeds.

The theory characterizes the replay-induced update distribution for a fixed buffer, priorities, and target parameters. It isolates conditional outcome selection as one component of learning dynamics, alongside priority refresh, coverage, exploration, optimization, and abstraction quality.

SAMPLE is the most practical variant. It is cheap, preserves outcome variability, and works with exact or latent groups. MODEL is strong in exact discrete environments because it samples a joint empirical outcome tuple, but it requires a faithful outcome table. AVG is theoretically natural and has lower conditional target variance for fixed targets. In FrozenLake, increasing the averaging subset does not consistently improve learning (Appendix E); the fixed-target variance calculation alone does not determine performance.

The value of sibling correction depends on the groups and their stochastic outcomes. If groups are mostly singletons, outcome variability is small, or large errors mainly reflect reducible underfitting, uniform sibling selection may offer little benefit. Approximate groups can also merge observations with different outcome laws or split observations that could share useful experience. The mixed Breakout and clean-MinAtar results motivate studying these conditions. Learning sibling abstractions and combining noise-aware group priorities with conditional outcome correction remain promising directions.

References

- [1] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016. URL <https://arxiv.org/abs/1606.01540>.
- [2] Rodrigo Antonio Carrasco-Davis, Sebastian Lee, Claudia Clopath, and Will Dabney. Uncertainty prioritized experience replay. *Reinforcement Learning Journal*, 6:585–623, 2025. URL <https://rlj.cs.umass.edu/2025/papers/Paper45.html>.
- [3] William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Will Dabney. Revisiting fundamentals of experience replay. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3061–3071. PMLR, 2020. URL <https://proceedings.mlr.press/v119/fedus20a.html>.
- [4] Scott Fujimoto, David Meger, and Doina Precup. An equivalence between loss functions and non-uniform sampling in experience replay. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/a3bf6e4db673b6449c2f7d13ee6ec9c0-Abstract.html.
- [5] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Gheshlaghi Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pages 3215–3222. AAAI Press, 2018. doi: 10.1609/aaai.v32i1.11796. URL <https://ojs.aaai.org/index.php/AAAI/article/view/11796>.
- [6] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, and David Silver. Distributed prioritized experience replay. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=H1Dy---OZ>.
- [7] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.
- [8] Steven Kapturowski, Georg Ostrovski, John Quan, Rémi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=r1lyTjAqYX>.
- [9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [10] Aviral Kumar, Abhishek Gupta, and Sergey Levine. DisCor: Corrective feedback in reinforcement learning via distribution correction. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/d7f426ccbc6db7e235c57958c21c5dfa-Abstract.html>.
- [11] Thibault Lahire, Matthieu Geist, and Emmanuel Rachelson. Large batch experience replay. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 11790–11813. PMLR, 2022. URL <https://proceedings.mlr.press/v162/lahire22a.html>.
- [12] Ang A. Li, Zongqing Lu, and Chenglin Miao. Revisiting prioritized experience replay: A value perspective. *arXiv preprint arXiv:2102.03261*, 2021. URL <https://arxiv.org/abs/2102.03261>.
- [13] Changling Li, Zhang-Wei Hong, Pulkit Agrawal, Divyansh Garg, and Joni Pajarinen. ROER: Regularized optimal experience replay. *Reinforcement Learning Journal*, 4:1598–1618, 2024. URL <https://rlj.cs.umass.edu/2024/papers/Paper198.html>.
- [14] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992. doi: 10.1007/BF00992699.

- [15] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- [16] Yangchen Pan, Jincheng Mei, Amir-massoud Farahmand, Martha White, Hengshuai Yao, Mohsen Rohani, and Jun Luo. Understanding and mitigating the limitations of prioritized experience replay. In James Cussens and Kun Zhang, editors, *Proceedings of the 38th Conference on Uncertainty in Artificial Intelligence*, volume 180 of *Proceedings of Machine Learning Research*, pages 1561–1571. PMLR, 2022. URL <https://proceedings.mlr.press/v180/pan22a.html>.
- [17] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016. URL <https://arxiv.org/abs/1511.05952>.
- [18] Shivakanth Sujit, Somjit Nath, Pedro Braga, and Samira Ebrahimi Kahou. Prioritizing samples in reinforcement learning with reducible loss. In *Advances in Neural Information Processing Systems*, volume 36, pages 23237–23258, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/48726631f87322012c6be38e00c72a47-Abstract-Conference.html.
- [19] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL <https://arxiv.org/abs/2407.17032>.
- [20] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, pages 6306–6315, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/7a98af17e63a0ac09ce2e96d03992fbc-Abstract.html.
- [21] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2094–2100. AAAI Press, 2016. doi: 10.1609/aaai.v30i1.10295. URL <https://ojs.aaai.org/index.php/AAAI/article/view/10295>.
- [22] Kenny Young and Tian Tian. MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments. *arXiv preprint arXiv:1903.03176*, 2019. URL <https://arxiv.org/abs/1903.03176>.
- [23] Shangdong Zhang and Richard S. Sutton. A deeper look at experience replay. *arXiv preprint arXiv:1712.01275*, 2017. URL <https://arxiv.org/abs/1712.01275>.

A Code and reproducibility

Code and experiment scripts are available at <https://github.com/oscar-omlf/sibling-aware-replay>.

B Additional derivations

B.1 Partial importance correction and the conditional exponent

We analyze one learning step to separate replay sampling from IS weighting. Hold the buffer, sibling groups, and positive priorities fixed, and average over replay sampling at the current network parameters. Gradients pass through predicted Q-values; priorities, IS weights, and Bellman targets are treated as constants. With $u_i = p_i^\alpha$ and unnormalized weights, transition i contributes to the expected gradient with coefficient

$$P(i)w_i = N^{-\beta} S^{\beta-1} u_i^{1-\beta}.$$

Normalizing these coefficients within group g gives the effective weights $\pi_\beta(i | g)$ in Equation (6) and mean target $\sum_{i \in I(g)} \pi_\beta(i | g) Y_i$. These weights describe the update; PER still samples sibling i with probability u_i/s_g . Summing the coefficients within a group gives its total weight in the expected update, up to a common factor:

$$\text{PER : } \sum_{i \in I(g)} u_i^{1-\beta}, \quad \text{SAMPLE : } n_g^\beta s_g^{1-\beta}.$$

PER and SAMPLE therefore select groups with the same probabilities at a fixed buffer, but can assign them different total update weights when $0 < \beta < 1$. At $\beta = 1$, both unnormalized estimators recover the uniform-buffer expected loss or semi-gradient. This single-step equality does not imply unbiased optimal values or identical Adam updates.

We use stratified sampling: divide the cumulative priority mass into equal intervals and sample once from each. This preserves the expected minibatch mean under PER. We then divide each IS weight by the largest weight in its minibatch. This sampled maximum generally changes the expectation above; at batch size one, for example, every normalized weight equals one regardless of β . Equation (6) thus describes unnormalized weights. For SAMPLE, weights and their normalization depend only on the selected groups, so they preserve uniform selection within each group.

B.2 Effective sampling distribution of SAMPLE

A transition j can be selected for training whenever the PER anchor lies in its group $g = \kappa(j)$. Multiplying the group probability s_g/S by the uniform sibling probability $1/n_g$ gives Equation (9); comparing this with the uniform-buffer probability $1/N$ gives the IS ratio in Equation (10).

With full, unnormalized IS correction and fixed priorities and losses, the expected weighted loss equals the uniform-buffer average. Partial correction leaves different weights across groups, but siblings remain uniform within each selected exact group.

B.3 Bias of PER's within-group target

For a fixed group g , let Y_i denote the Bellman target of sibling i . The empirical conditional target is

$$\bar{Y}_g = \frac{1}{n_g} \sum_{i \in I(g)} Y_i,$$

whereas PER without IS correction ($\beta = 0$) uses the priority-weighted conditional mean

$$\tilde{Y}_g^{\text{PER}} = \sum_{i \in I(g)} \frac{u_i}{s_g} Y_i.$$

The finite-buffer within-group bias is therefore

$$B_g^{\text{PER}} = \tilde{Y}_g^{\text{PER}} - \bar{Y}_g = \sum_{i \in I(g)} \left(\frac{u_i}{s_g} - \frac{1}{n_g} \right) Y_i.$$

This term is zero if priorities are constant within the group or if the priority deviations are uncorrelated with sibling targets in the finite buffer. It can be large when rare stochastic outcomes have systematically larger TD errors. This is the sampling-target bias ($\beta = 0$); for unnormalized partial correction, replace u_i/s_g by $\pi_\beta(i | g)$.

B.4 Variance of SAMPLE and AVG

For a selected group g , hold its Bellman targets fixed and consider the randomness of sibling selection. SAMPLE draws one sibling uniformly, so its target variance is the variance across the stored sibling targets. If AVG averages K independent sibling targets sampled with replacement, then

$$\text{Var}(\hat{Y}_{g,K} | g) = \frac{\sigma_g^2}{K},$$

where σ_g^2 is the variance of a uniformly sampled sibling target. If it samples without replacement, the finite-population correction gives

$$\text{Var}(\hat{Y}_{g,K} | g) = \frac{\sigma_g^2}{K} \left(1 - \frac{K-1}{n_g-1} \right), \quad K \leq n_g, \quad n_g > 1.$$

A singleton group has zero target variance under both methods. Thus AVG has the same conditional mean as SAMPLE and no greater fixed-target variance. This calculation alone does not determine learning performance: target networks, stored priorities, and optimizer dynamics also evolve during training.

B.5 MODEL targets

MODEL counts occurrences of each complete outcome tuple

$$Z = (s', r, d_{\text{term}}, d_{\text{trunc}})$$

for each exact (s, a) group in the current buffer. It samples a tuple in proportion to its count, preserving dependencies between rewards, next states, and terminations. Sampling a next state separately from an averaged reward can create an outcome absent from the buffer, rather than sampling from its empirical conditional distribution.

B.6 Huber loss and mean-value estimation

Sampling correction and loss choice affect different parts of an update [4]. Consider fitting a single value q to OutlierBandit's risky rewards: $R = 100$ with probability 0.01 and $R = 0$ otherwise. Weight these outcomes by their true probabilities, without priority reweighting. Squared loss has minimizer $\mathbb{E}[R] = 1$. Huber loss H_c [7] has derivative $H'_c(e) = \text{clip}(e, -c, c)$. For threshold $c = 1$ and $0 \leq q \leq 1$,

$$\frac{d}{dq} \mathbb{E}[H_1(q - R)] = 0.99q - 0.01.$$

The convex expected loss therefore has unique minimizer $q_H = 1/99 \approx 0.0101$.

Huber bounds gradient contributions without changing outcome frequencies; its optimum can differ from the mean for asymmetric targets. The discrepancy depends on the threshold relative to reward scale. Uniform sibling selection recovers empirical outcome frequencies, but choosing Huber can still shift the fitted value away from their mean. Both squared-loss and Huber optima here prefer the safe action's deterministic reward 2, so this example concerns value estimation rather than policy failure.

C Experimental details

C.1 Priority updates

After each learning step, we update stored priorities using TD errors computed before the optimizer step, with the current online and target networks. We assign $p_i = |\delta_i| + 10^{-6}$ and sample using masses p_i^α . New entries receive the largest raw priority seen so far (initially one); entries otherwise

keep their priorities until updated. Group masses s_g track the sum of the current entries’ masses, including insertions and evictions.

The *anchor* is the transition initially selected by PER. PER trains on it and updates its priority with its own TD error. SAMPLE trains on a uniformly selected sibling, which can be the anchor, and updates only that sibling’s priority.

MODEL samples a complete $(s', r, d_{\text{term}}, d_{\text{trunc}})$ outcome from its group’s buffer counts. It updates the anchor’s priority with the TD error for this sampled outcome, which may differ from the stored one. For PER, SAMPLE, and MODEL, repeated assignments to an entry follow minibatch order: the last assignment wins.

AVG averages all sibling targets or, when capped, a uniform subset sampled without replacement. The anchor is eligible for the selection but is not always included. The priority is the absolute TD error against the averaged target, plus 10^{-6} . With `update_all_siblings=false`, only the anchor receives it; with the flag true, every sibling used in the average receives it. If several AVG updates address the same entry, the largest proposed priority wins.

OutlierBandit AVG uses full groups with the flag false; TwoChains AVG uses full groups with the flag true; FrozenLake AVG-g2/g4 averages at most two/four siblings with the flag true. These rules allow group priorities to evolve differently across methods.

Loss and optimizer. All DQN variants use Double-DQN targets [21], Adam [9], and mean-squared TD error. MSE retains sensitivity to large residuals, whereas Huber loss [7] bounds the derivative with respect to the scalar TD error. We hold MSE fixed across methods; the empirical comparison therefore concerns this loss, while the fixed-buffer sampling identities do not depend on it.

MinAtar settings and learning rates. For MinAtar [22], we use two convolutional layers with 16 and 32 channels and a fully connected layer with 128 units. We train for 5 million environment steps with $\gamma = 0.99$, sticky-action probability 0.1, replay capacity 100,000, batch size 32, and target-network updates every 1000 steps. Epsilon decreases from 1 to 0.1 over 100,000 steps. PER and SAMPLE use $\alpha = 0.6$ and anneal β from 0.4 toward 1 over 5 million replay updates.

We select a learning rate for each game and method using AUC on clean games, then use it for both clean and MP-TAIL training. We test $1.2 \cdot 10^{-4}$, $6 \cdot 10^{-5}$, $3 \cdot 10^{-5}$, and $7.5 \cdot 10^{-6}$, plus $3.75 \cdot 10^{-6}$ and $1.875 \cdot 10^{-6}$ for PER. Table 3 lists the selected rates. Each reported setting uses five seeds. This comparison measures how well learning rates chosen on clean games transfer to reward tails. For MinAtar AUC differences, unadjusted paired 95% t intervals use four degrees of freedom and assume approximately normal seed differences. Intervals for all four positive mean differences over PER exclude zero; differences over DDQN include zero except in Freeway, whose interval is $[-13.19, -7.95]$.

Table 3: Selected MinAtar learning rates used for the reported clean and MP-TAIL runs.

Game	DDQN lr	PER lr	SAMPLE lr
Asterix	7.5e-06	1.88e-06	7.5e-06
Breakout	6e-05	3e-05	6e-05
Freeway	7.5e-06	1.88e-06	7.5e-06
Seaquest	7.5e-06	3.75e-06	7.5e-06
SpaceInv.	3e-05	7.5e-06	3e-05

FrozenLake rewards and transitions. In FrozenLake-H100/H300, the intended action occurs with probability 0.99; the two side slips share the remaining probability. Rewards are +100 at the goal, −300 at holes, and −0.01 per step. H50 uses +50, −100, and −0.01. Groups are exact (s, a) pairs. Under the known transition law and infinite-horizon optimal values, with $\gamma = 0.99$, $\alpha = 0.6$, and no IS correction, the largest downward target shifts among visited greedy state-action pairs with hole risk are approximately −50.4 and −15.3, respectively. These are analytical examples using the stated reference values.

C.2 Fixed- α/β sweep: settings and complete results

We train PER and SAMPLE using Double DQN, two hidden layers of 64 ReLU units, Adam, MSE, gradient-norm clipping at 10, batch size 32, and one gradient step per environment step. Both methods use exact groups, add 10^{-6} to absolute TD errors for priorities, and divide IS weights by their minibatch maximum. We stop bootstrapping at time limits. In each cell, the methods use the same training settings and seeds, with fixed α and β as listed in Section 5.3.

Table 4: Training and evaluation settings for the fixed- α/β sweep. Evaluation intervals count training episodes; all other intervals count environment steps.

Setting	OutlierBandit	TwoChains	FrozenLake-H100/H300
Training steps	50,000	100,000	500,000
Replay capacity	5,000	5,000	50,000
Learning starts (steps)	500	1000	5,000
Learning rate	7.5×10^{-4}	7.5×10^{-4}	3×10^{-3}
Discount γ	0	0.99	0.99
Target update interval	500	1000	1000
Initial ϵ	1	1	1
Final ϵ	0.02	0.05	0.05
ϵ decay steps	10,000	50,000	500,000
Evaluation episodes	500	200	50
Evaluation interval	500	100	20

Table 4 gives the environment-specific settings. TwoChains has a safe-chain length of 10 (eleven steps to reward), risky success probability 0.01, risky reward 80, and episode limit 20. Evaluation is scheduled by episode count, so paired methods can be evaluated at different step counts; each run also has a final evaluation. The main exact-environment comparisons use the same training and evaluation settings, with $\alpha = 0.6$ and β increasing from 0.4 toward 1 over 50k replay updates in Bandit, 100k in TwoChains, and 500k in FrozenLake. H50 uses the FrozenLake settings with its milder rewards. These comparisons use seeds 0–9 in Bandit and TwoChains and 1–10 in FrozenLake.

Normalized return AUC is the main metric. Late return and success average evaluations in the final 25k steps. We subtract PER from SAMPLE at each cell and seed, then compute means and uncertainty across seeds. Tables 5–7 report all 48 cells with paired seeds 0–9. The sweep includes 960 runs and 480 paired differences. The outcome diagnostics use these same runs.

To account for the 48 comparisons, we compute 95% Bonferroni simultaneous t intervals for paired return-AUC differences, with 9 degrees of freedom and critical value $t_{9,1-0.05/(2 \cdot 48)}$. These assume approximately normal seed differences. The paired-difference columns report these simultaneous intervals. Figures show descriptive means and SE across seeds; we do not select a best configuration. The sweep holds β fixed, whereas Table 1 uses annealed β .

C.3 Diagnostic definitions

Performance Metrics. AUC measures learning efficiency over the training horizon T . We integrate the evaluation curve by the trapezoidal rule from the first recorded evaluation and divide by T ; we do not add an unobserved time-zero evaluation. Late-window metrics average evaluation checkpoints near the end of training, with windows specified below.

Variation in importance weights. IS-weight ESS is $(\sum_b w_b)^2 / \sum_b w_b^2$. For batch size B , weight mean μ_w , and population standard deviation σ_w , it equals $B\mu_w^2/(\mu_w^2 + \sigma_w^2)$. Multiplying all weights by a common factor leaves it unchanged. At $\beta = 0$, every weight is one and ESS equals B , whatever the sampling distribution. ESS therefore measures weight variation, not sampling concentration or target bias.

Outcome frequencies. We compare how often an outcome appears among a group’s stored transitions, $f_g(o)$, with its probability under replay, $q_g(o)$. Entries with the same complete outcome tuple are counted together. Their total variation distance D_g^{out} (Equation (15)) is the probability mass that must be reassigned between outcomes to make the distributions agree. It

Table 5: OutlierBandit fixed grid, seeds 0–9. PER and SAMPLE: mean $\pm$ SE. Paired SAMPLE minus PER: mean [95% simultaneous CI across all 48 comparisons].

α	β	PER AUC	SAMPLE AUC	Difference [95% CI]
0.1	0	1.978 ± 0.001	1.978 ± 0.001	0.000 [0.000, 0.000]
0.1	0.4	1.974 ± 0.003	1.976 ± 0.002	0.002 [−0.007, 0.011]
0.1	0.7	1.961 ± 0.006	1.974 ± 0.002	0.012 [−0.017, 0.042]
0.1	1	1.973 ± 0.003	1.973 ± 0.001	0.000 [−0.018, 0.018]
0.3	0	1.341 ± 0.107	1.977 ± 0.002	0.636 [0.129, 1.143]
0.3	0.4	1.866 ± 0.020	1.976 ± 0.002	0.109 [0.014, 0.205]
0.3	0.7	1.934 ± 0.011	1.974 ± 0.003	0.041 [−0.009, 0.090]
0.3	1	1.960 ± 0.004	1.965 ± 0.005	0.005 [−0.016, 0.026]
0.6	0	1.003 ± 0.011	1.977 ± 0.001	0.974 [0.921, 1.027]
0.6	0.4	1.019 ± 0.017	1.973 ± 0.003	0.954 [0.871, 1.036]
0.6	0.7	1.714 ± 0.062	1.974 ± 0.003	0.259 [−0.038, 0.557]
0.6	1	1.977 ± 0.001	1.970 ± 0.004	−0.007 [−0.025, 0.010]
1	0	1.003 ± 0.011	1.978 ± 0.001	0.975 [0.923, 1.027]
1	0.4	1.003 ± 0.011	1.976 ± 0.002	0.973 [0.920, 1.027]
1	0.7	1.033 ± 0.023	1.975 ± 0.002	0.942 [0.832, 1.052]
1	1	1.974 ± 0.003	1.975 ± 0.002	0.000 [−0.010, 0.011]

Table 6: TwoChains fixed grid ($p_{\text{success}} = 0.01$, $r_{\text{high}} = 80$), seeds 0–9. PER and SAMPLE: mean $\pm$ SE. Paired SAMPLE minus PER: mean [95% simultaneous CI across all 48 comparisons].

α	β	PER AUC	SAMPLE AUC	Difference [95% CI]
0.1	0	0.904 ± 0.012	0.957 ± 0.007	0.053 [0.001, 0.105]
0.1	0.4	0.923 ± 0.009	0.948 ± 0.007	0.025 [−0.006, 0.056]
0.1	0.7	0.926 ± 0.008	0.949 ± 0.004	0.024 [−0.003, 0.050]
0.1	1	0.949 ± 0.006	0.942 ± 0.008	−0.006 [−0.027, 0.015]
0.3	0	0.810 ± 0.009	0.955 ± 0.006	0.145 [0.104, 0.186]
0.3	0.4	0.812 ± 0.009	0.949 ± 0.006	0.137 [0.108, 0.166]
0.3	0.7	0.888 ± 0.015	0.941 ± 0.008	0.053 [−0.005, 0.111]
0.3	1	0.947 ± 0.005	0.947 ± 0.004	−0.000 [−0.025, 0.025]
0.6	0	0.806 ± 0.008	0.953 ± 0.004	0.147 [0.115, 0.179]
0.6	0.4	0.807 ± 0.009	0.935 ± 0.006	0.128 [0.090, 0.165]
0.6	0.7	0.829 ± 0.014	0.942 ± 0.008	0.113 [0.061, 0.164]
0.6	1	0.937 ± 0.007	0.938 ± 0.008	0.001 [−0.024, 0.025]
1	0	0.804 ± 0.008	0.961 ± 0.005	0.156 [0.129, 0.184]
1	0.4	0.807 ± 0.009	0.945 ± 0.007	0.137 [0.104, 0.171]
1	0.7	0.811 ± 0.008	0.940 ± 0.006	0.129 [0.097, 0.161]
1	1	0.943 ± 0.008	0.942 ± 0.009	−0.002 [−0.021, 0.017]

ranges from zero to one. With fixed Bellman targets, the target range bounds the resulting mean shift: $|B_g^{\text{PER}}| \leq (\max_i Y_i - \min_i Y_i) D_g^{\text{out}}$. In Bandit’s risky arm, this shift is exactly $100[q_g(100) - f_g(100)]$. Uniform sibling selection has zero outcome TV and zero shift relative to its buffer. These quantities describe selection before IS weighting.

For Bandit PER at fixed $(\alpha, \beta) = (0.6, 0.4)$, we measure the buffer every 1000 training steps using seeds 0–9 of the sweep. Averaging 50 checkpoints per seed gives risky-arm outcome TV 0.06197 ± 0.00090 and unweighted target shift 6.197 ± 0.090 reward units. Unnormalized partial IS reduces this shift to 2.378 ± 0.065 , using coefficients $u_i^{1-\beta}$. This calculation describes unnormalized weights; the implemented normalization is examined below.

Measurements across the parameter sweep. For both methods in all 48 cells, we measure the full buffer after sampling and before optimization or priority updates: every 1000 steps in Bandit and TwoChains and every 2000 steps in FrozenLake, once learning starts. We record counts, priority masses, sampling probabilities, and Double-DQN targets for each joint outcome $(s', r, d_{\text{term}}, d_{\text{trunc}})$. Logging makes no random draws or changes to learning. Performance and diagnostics come from the same ten runs per method and cell.

Table 7: FrozenLake-H100/H300 fixed grid, seeds 0–9. PER and SAMPLE: mean $\pm$ SE. Paired SAMPLE minus PER: mean [95% simultaneous CI across all 48 comparisons].

α	β	PER AUC	SAMPLE AUC	Difference [95% CI]
0.1	0	8.864 $\pm$ 3.255	9.594 $\pm$ 4.138	0.730 [−21.407, 22.868]
0.1	0.4	15.055 $\pm$ 3.014	11.089 $\pm$ 3.301	−3.966 [−28.780, 20.849]
0.1	0.7	11.055 $\pm$ 3.557	14.208 $\pm$ 3.824	3.153 [−24.418, 30.725]
0.1	1	11.694 $\pm$ 3.086	16.151 $\pm$ 3.223	4.457 [−23.250, 32.163]
0.3	0	6.051 $\pm$ 2.395	12.832 $\pm$ 4.006	6.780 [−14.535, 28.096]
0.3	0.4	10.281 $\pm$ 4.391	6.310 $\pm$ 4.108	−3.971 [−29.898, 21.957]
0.3	0.7	6.417 $\pm$ 1.744	16.236 $\pm$ 3.095	9.818 [−4.034, 23.670]
0.3	1	8.370 $\pm$ 4.086	8.464 $\pm$ 3.646	0.094 [−22.551, 22.739]
0.6	0	−13.257 $\pm$ 2.126	8.175 $\pm$ 2.419	21.432 [12.958, 29.906]
0.6	0.4	−10.655 $\pm$ 2.428	10.709 $\pm$ 4.272	21.364 [−5.751, 48.479]
0.6	0.7	−9.196 $\pm$ 2.468	5.787 $\pm$ 3.603	14.983 [−7.481, 37.448]
0.6	1	2.755 $\pm$ 1.715	4.654 $\pm$ 3.958	1.899 [−17.070, 20.868]
1	0	−25.633 $\pm$ 1.937	−6.380 $\pm$ 3.910	19.252 [2.714, 35.790]
1	0.4	−23.983 $\pm$ 1.273	−9.271 $\pm$ 3.692	14.712 [−5.889, 35.313]
1	0.7	−20.707 $\pm$ 1.505	−9.866 $\pm$ 3.668	10.841 [−7.892, 29.574]
1	1	−4.210 $\pm$ 2.870	−13.782 $\pm$ 3.536	−9.571 [−33.657, 14.514]

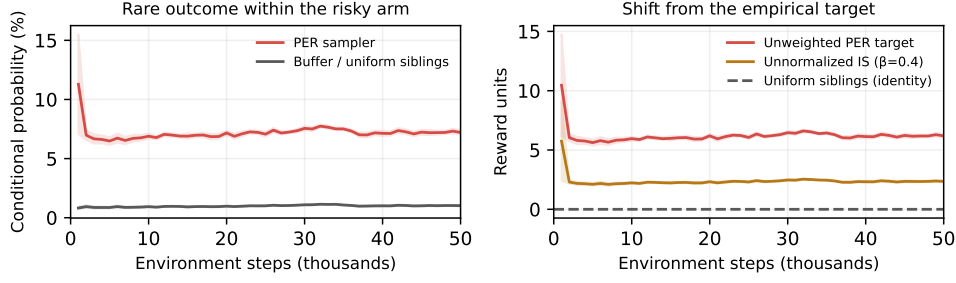


Figure 6: Bandit conditional outcome diagnostics at fixed $\alpha = 0.6$, $\beta = 0.4$, mean $\pm$ SE over ten seeds. Left: PER’s rare-outcome probability versus its buffer frequency. Right: target shifts relative to the empirical mean; the IS curve uses unnormalized weights, not the implemented minibatch-max normalization. Uniform siblings on this same buffer give zero shift by identity.

For each group, we measure how replay shifts the mean target away from the empirical mean $\bar{Y}_g$. For PER, this shift is $\Delta_g = \sum_o q_g(o)Y_g(o) - \bar{Y}_g$; after unnormalized IS weighting, $\Delta_{g,\beta}$ uses Equation (6). SAMPLE retains $f_g(o)$ before and after group-aware weighting, so both conditional shifts are zero.

We average across groups using their selection probabilities s_g/S : $\sum_g (s_g/S) D_g^{\text{out}}$ measures outcome distortion, and $\sum_g (s_g/S) |\Delta_g|$ measures target-shift magnitude. Absolute values prevent shifts in opposite directions from cancelling. We use the same group weights for $\Delta_{g,\beta}$ to isolate the within-group effect of IS. We first average checkpoints within each seed, then report mean and SE across seeds.

Targets after weight normalization. For each training minibatch, we record outcome counts, normalized weight sums, and each target’s deviation from its group’s empirical mean, $Y_b - \bar{Y}_{g_b}$. Using the actual normalized weights $\hat{w}$, the average across logged steps t and batch entries b is

$$\frac{\sum_{t,b} \hat{w}_{t,b} (Y_{t,b} - \bar{Y}_{g_b,t})}{\sum_{t,b} \hat{w}_{t,b}}.$$

This signed average includes sampling noise and allows shifts in different groups to cancel. It describes target deviations rather than the full parameter update. We log the full-buffer and training-minibatch target evaluations separately. For SAMPLE, weights are constant within each selected group, so normalization preserves conditional uniformity.

Table 8 compares the absolute conditional shifts with the signed measurements under the implemented weights. The latter allow cancellation across groups, so their magnitudes can differ; neither metric alone determines learning performance.

Table 8: Target shifts at $\alpha = 0.6$, $\beta = 0.4$, mean $\pm$ SE over ten seeds. Conditional columns compare group-weighted absolute shifts before and after unnormalized IS; sampled columns use signed target deviations and actual minibatch-normalized weights. SAMPLE’s conditional shifts are zero.

Environment	PER conditional shift (absolute)		Sampled shift (signed)	
	Before IS	After IS	PER	SAMPLE
OutlierBandit	6.14 ± 0.09	2.36 ± 0.06	2.43 ± 0.11	-0.0003 ± 0.0200
TwoChains	4.09 ± 0.06	1.55 ± 0.03	1.30 ± 0.06	-0.0007 ± 0.0079
FrozenLake	9.61 ± 0.19	3.58 ± 0.07	-0.67 ± 0.08	-0.048 ± 0.056

Sampling individual buffer entries. This diagnostic measures unequal sampling of individual buffer entries within a group, even when those entries share the same outcome. For PER’s conditional probabilities $q_g(i) = u_i/s_g$, define

$$C_g = \sum_{i \in I(g)} \frac{(q_g(i) - 1/n_g)^2}{1/n_g} = n_g \sum_{i \in I(g)} q_g(i)^2 - 1, \quad C = \sum_g \frac{s_g}{S} C_g.$$

Uniform selection gives $C_g = 0$; concentrating probability on fewer entries increases it. Equivalently, $C_g = n_g/n_{\text{eff},g} - 1$, where $n_{\text{eff},g} = 1/\sum_i q_g(i)^2$ is the effective number of sampled entries. This measures concentration within groups, separately from group allocation.

The ratio $R_i = n_g u_i/s_g$ compares an entry’s conditional PER probability with uniform sibling sampling. We log its minibatch mean and population standard deviation before priority updates. Under PER, $\mathbb{E}[R_i | g] = 1 + C_g$, so the logged mean minus one estimates C , also with stratified sampling. Finite-sample estimates can be negative; we leave them unclipped. We average checkpoints within runs, then compute means and SE across seeds.

Under SAMPLE, the trained sibling is uniform, so the actual conditional divergence is zero. The logged ratio still uses PER probabilities: it describes how PER would sample from SAMPLE’s current buffer. Its mean has expectation one, while $\sigma_R^2 + (\mu_R - 1)^2$ estimates that hypothetical PER divergence, using the logged mean μ_R and standard deviation σ_R . For AVG and MODEL, the logged ratio describes anchor priorities; their targets are averaged or resampled.

Unequal entry probabilities do not necessarily distort outcome frequencies. At step 41k of Bandit PER seed 0 with annealed β , the buffer contains 4953 safe-reward transitions and 47 zero-reward risky transitions, with no rare reward. Each group has only one outcome, so $D_g^{\text{out}} = 0$, yet the logged entry-divergence estimate is 4207.8 and all sampled TD errors are zero. Insertion and priority updates can leave identical outcomes with different priorities. Outcome TV ignores this entry-level variation. For fixed targets, Cauchy–Schwarz gives $(B_g^{\text{PER}})^2 \leq C_g \text{Var}_{U_g}(Y)$; a positive upper bound does not establish bias.

We average these descriptive statistics over the final $\max(25k, 0.2T)$ environment steps: 25k for OutlierBandit and TwoChains, 100k for FrozenLake, and 1M for MinAtar. TD-error standard deviation describes signed residuals in sampled minibatches, not error against the true value function; averaging targets can reduce it directly. Action L1 compares window averages of *cumulative* behavior and replay action frequencies, not instantaneous sampling probabilities.

C.4 Other replay diagnostics

Figure 9 shows IS-weight ESS, sampled TD-error standard deviation, and the L1 difference between cumulative behavior and replay action frequencies. SAMPLE generally has less variable weights and smaller sampled TD errors under MP-TAIL. The cumulative action counts reflect both past behavior and buffer turnover; they do not measure the current group-selection probabilities in Equation (9).

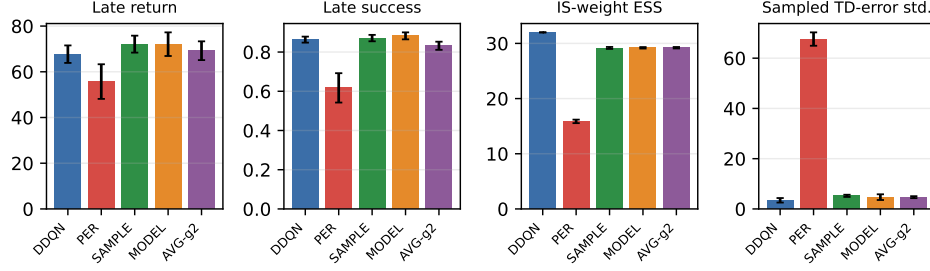


Figure 7: FrozenLake-H100/H300, mean $\pm$ SE over ten seeds. Return and success average the final 25k steps; IS-weight ESS and sampled TD-error dispersion average the final 100k. Return and success summarize late-window learning performance.

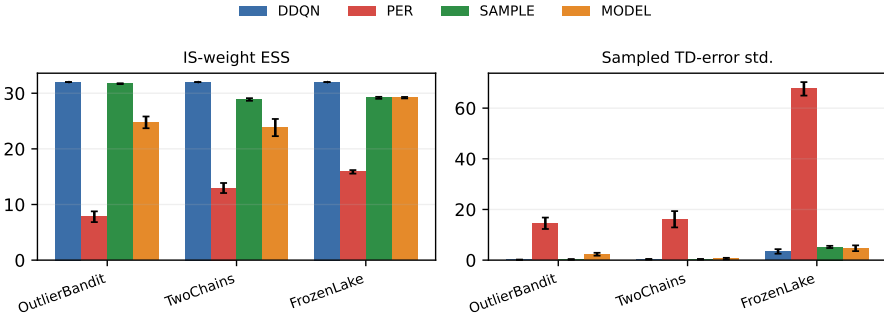


Figure 8: Exact-group descriptive diagnostics, mean $\pm$ SE over ten seeds. IS-weight ESS measures weight dispersion; sampled TD-error standard deviation depends on the sampled or constructed targets. Windows follow Appendix C.3.

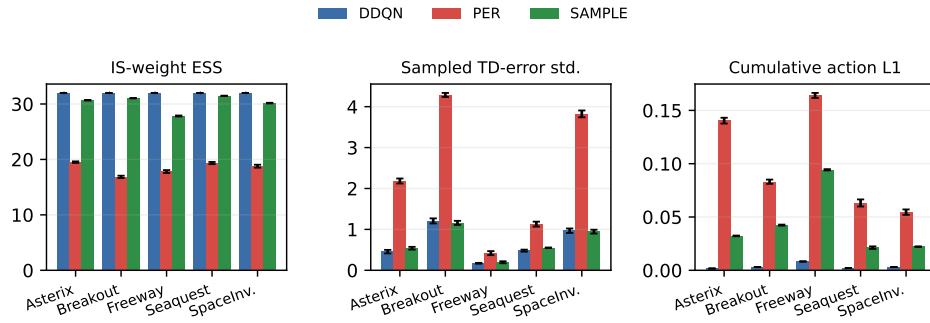


Figure 9: MinAtar MP-TAIL descriptive diagnostics, mean $\pm$ SE over five seeds, final 20% of training. ESS measures IS-weight dispersion; action L1 compares late-window averages of cumulative behavior and replay frequencies.

D VQ-VAE grouping

We train a separate VQ-VAE for each MinAtar game and freeze its encoder before RL training. The encoder maps an observation to a discrete code grid. Observations with the same code grid and action share an approximate sibling group, $(\phi(s), a)$. The VQ-VAE defines these groups and is not trained jointly with the RL objective. We tune the reconstruction loss (MSE or binary cross-entropy, BCE), code-grid size, and codebook size using reconstruction and grouping diagnostics. The selected encoders use BCE.

We choose the abstraction size to provide enough siblings per group while retaining distinctions between states. We compare code grids of 1×1 and 2×1 and codebooks with 8, 16, 32, and 256

entries where relevant. Coarse codes can merge states with different outcome distributions; fine codes can leave too few siblings for useful replacement. The selected 2×1 , $K = 16$ codes provide useful group sizes in four games. SpaceInvaders uses $K = 32$ to retain more state distinctions while keeping enough siblings. Table 9 reports group sizes and code-action coverage.

Table 9: Selected VQ-VAE grouping statistics. Recon.: final logged training-minibatch BCE; perp.: exponentiated code-use entropy; pair cov.: fraction of possible code-action buckets observed. Median bucket counts transitions per occupied bucket; frac ≥ 100 is the fraction of sampled transitions in buckets containing at least 100 transitions.

Game	Grid, K	recon.	perp.	pair cov.	median bucket	frac ≥ 100
Asterix	$2 \times 1, 16$	0.061	14.0	0.664	38	0.822
Breakout	$2 \times 1, 16$	0.028	8.7	0.133	156	0.973
Freeway	$2 \times 1, 16$	0.090	15.1	0.951	33	0.677
Seaquest	$2 \times 1, 16$	0.024	12.1	0.569	44	0.767
SpaceInv.	$2 \times 1, 32$	0.082	16.8	0.158	41	0.760

E Control and robustness results

Clean MinAtar. Table 10 summarizes the clean-training curves in Figure 3. PER has higher mean AUC than DDQN in Breakout and Seaquest, whereas DDQN leads in Asterix, Freeway, and SpaceInvaders. SAMPLE’s relative performance also varies by game. These controls distinguish the response to stochastic reward tails from differences already present under clean training.

Table 10: Clean MinAtar AUC return, mean $\pm$ standard error over five seeds.

Game	DDQN	PER	SAMPLE
Asterix	23.70 ± 0.46	22.97 ± 1.34	23.39 ± 0.50
Breakout	17.23 ± 0.62	19.17 ± 0.35	19.24 ± 0.51
Freeway	52.97 ± 0.15	49.80 ± 0.34	51.70 ± 0.11
Seaquest	32.63 ± 2.11	34.02 ± 2.27	35.57 ± 1.84
SpaceInv.	83.39 ± 3.02	67.92 ± 3.30	79.18 ± 1.45

Milder FrozenLake rewards. Figure 10 compares the two reward schedules. In H50, sibling-aware methods have higher mean AUC than PER, but DDQN has the highest mean AUC. Thus, recovery relative to PER also occurs with milder rewards, without implying an advantage over uniform replay. AVG-g4 has slightly higher mean AUC than AVG-g2 in H100/H300 (3.21 versus 2.64), but lower mean AUC in H50 (9.90 versus 13.47); increasing the averaging subset therefore does not consistently improve learning.

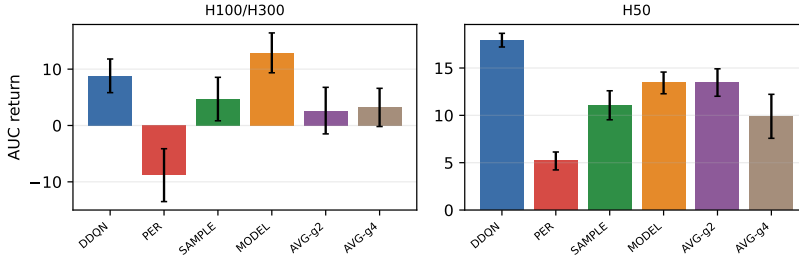


Figure 10: FrozenLake AUC return for both selected reward schedules, mean $\pm$ SE over ten seeds, including AVG-g2 and AVG-g4. H100/H300 is the sharper exact-state stress test; H50 is a milder robustness setting.